

CS599 大作业报告

Mailflow Agent: 多 Agent 邮件与日程助理系统

项目	内容
课程名称	企业级应用软件设计与开发
项目名称	Mailflow Agent
方向	方向一：Agentic AI 原生开发
学号	2025303048
姓名	黎鸿
专业	计算机技术 / 软件工程
指导教师	戚欣
提交日期	2026 年 6 月 22 日
GitHub 仓库	https://github.com/leehong020/cs599-project

目录

[一、选题背景与设计思想](#)

[二、Specs 规格文档](#)

[三、系统架构与设计](#)

[四、关键实现与代码展示](#)

[五、测试与评估](#)

[六、系统升级与扩展](#)

[七、课程总结](#)

一、选题背景与设计思想

1.1 问题定义

Mailflow Agent 面向邮件、日程和个人工作流场景，将 Gmail、Google Calendar、文件上下文、联系人、签名和记忆整合到一个多 Agent 系统中。用户用自然语言提出目标，系统通过 Supervisor、Context、Mail、Calendar、Review、Confirmation、Executor 等节点完成意图识别、字段补全、草稿生成、确认授权和真实工具执行。

1.2 现有方案不足

- 传统 Gmail 和 Calendar 客户端以页面操作为主，用户仍需手工拆解任务、复制上下文、核对收件人和时间字段。
- 通用聊天助手缺少项目内的工具链、状态管理和授权边界，容易停留在文本建议，不能稳定完成真实工作流。
- 普通 API 脚本缺少多轮追问、联系人解析、文件上下文、记忆和 Proposal + Authorization 安全闭环。

1.3 项目价值

项目选择方向一“Agentic AI 原生开发”，价值在于把大模型从问答接口变成可协作、可追踪、可授权的软件执行组件。系统不仅生成邮件或日程文本，还围绕工作项、草稿、Proposal、Authorization 和 Action Event 建立安全边界，使 AI 能参与真实操作而不绕过用户确认。

1.4 技术路线

- 前端使用 Vue 3 / Vite 构建聊天页、Gmail 页、Calendar 页和设置页；后端使用 FastAPI 提供认证、会话、工具、工作流和记忆接口。
- 核心编排使用 LangGraph / LangChain 风格的多 Agent 状态流，由 Supervisor 负责意图识别和 Agent 路由。
- 工具层封装 Gmail API、Google Calendar API、文件解析、联系人、签名和记忆能力，外部写操作进入 Proposal + Authorization 闭环。
- 数据层使用 SQLite / SQLAlchemy / Alembic 保存工作流事件、设置、联系人和记忆；.env 与 google_oauth.json 仅本地保存，不提交到 GitHub。

二、Specs 规格文档

2.1 Product Spec

模块	规格说明
目标用户	需要用自然语言统一处理 Gmail、Google Calendar、文件上下文和个人偏好的用户。
核心任务	邮件读取/草稿/发送确认，日历读取/创建/更新/删除确认，文件上传解析，联系人、签名和记忆辅助。
交互原则	用户提出目标；Agent 补全字段并生成 Artifact；外部写操作必须经过 Proposal 与 Authorization。
安全要求	API Key、OAuth Client Secret、token、.env、google_oauth.json 和 data/ 运行数据不进入仓库。
完成标准	本地可运行，已整理 README、架构说明、规格文档、验收矩阵和课程报告，满足最终提交状态。

2.2 Architecture Spec

层次	组件	职责
表现层	Vue 3 / Vite	聊天页、Gmail 页、Calendar 页、设置页和用户可见确认界面。
服务层	FastAPI	统一 API、OAuth 回调、Gmail/Calendar 路由、文件与记忆接口。
编排层	Graph nodes	加载状态、Supervisor 路由、Agent 执行、确认门、响应生成。
工具层	Tool functions	封装 Gmail、Calendar、文件、联系人、签名和记忆工具，

		统一返回 JSON 结果。
数据与配置层	SQLite / .env / google_oauth.json	保存运行状态、工作流事件和本地配置；敏感凭据不入库不入仓。

2.3 API Spec

API 组	典型接口	说明
assistant_graph	聊天/工作流入口	接收用户消息并驱动 Agent 状态流。
auth	Google OAuth 登录与回调	完成用户授权并保存本地访问令牌。
gmail	Gmail 草稿与邮件接口	读取、创建、更新、发送或删除邮件相关资源。
calendar	日历事件接口	创建、更新、删除或查询 Google Calendar 事件。
memory/settings/files	记忆、配置、文件接口	支撑上下文补全、用户偏好与附件处理。

三、系统架构与设计

系统总体架构

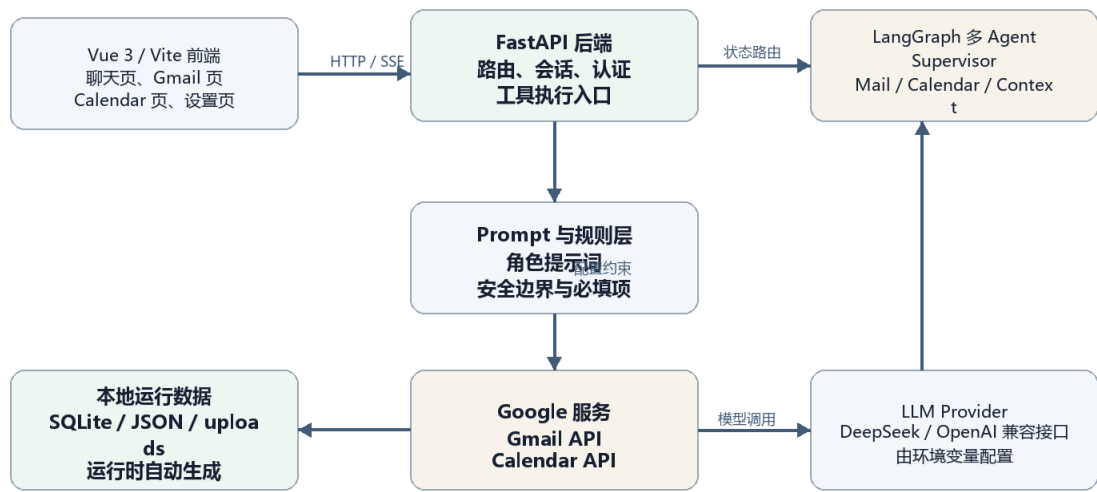


图 1 系统总体架构：前端、后端、Graph 编排、工具层与外部服务的关系

3.1 核心架构

系统采用前后端分离与 Agent 编排分层设计。Vue 3 前端负责聊天、Gmail、Calendar、设置和用户确认，FastAPI 后端统一承接 API、OAuth、工作流和工具入口，Graph 节点维护任务流转状态，工具层以受控函数形式访问 Gmail、Calendar、文件、联系人和记忆能力。模型提供理解和生成能力，但不直接持有凭据，也不绕过确认门执行写操作。

Agent 交互流程

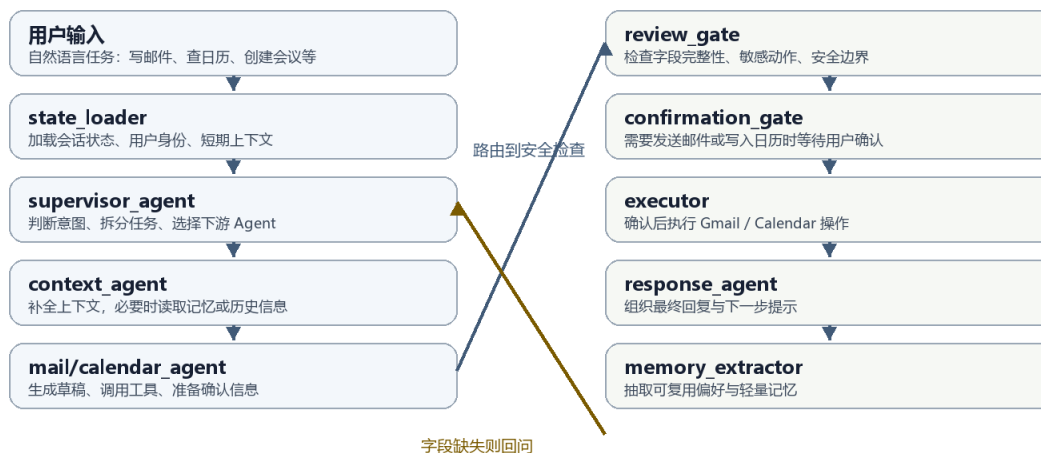
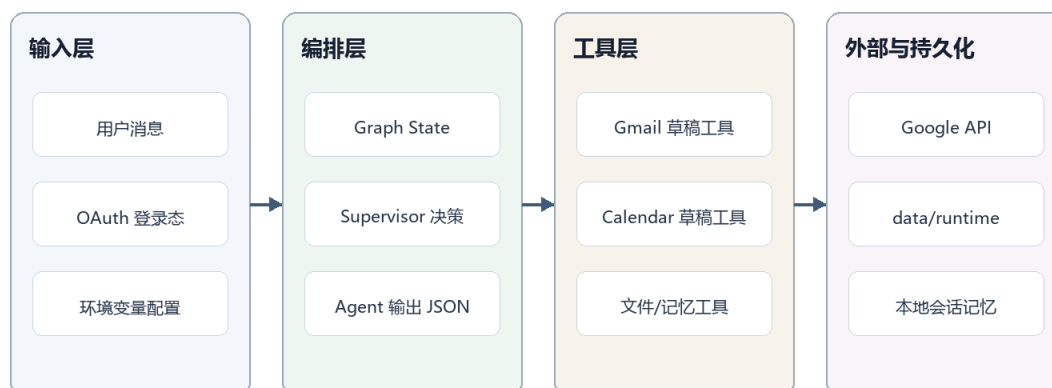


图 2 Agent 交互流程：从用户输入到安全检查、确认执行和记忆抽取

3.2 Agent 交互流程

Supervisor 是任务入口，先判断任务类型和所需字段，再把任务交给邮件、日历或上下文 Agent。Review Gate 与 Confirmation Gate 负责检查必填项和敏感动作；如果信息不足，系统返回追问；如果需要发送邮件或写入日历，系统先展示草稿，等待用户确认后再进入 executor。

数据流设计



敏感凭据不进入 Git: .env 与 google_oauth.json 仅本地保存, README 提供自行配置路径。

图 3 数据流设计: 输入、编排、工具执行和本地运行数据的边界

3.3 数据流设计

用户输入首先进入会话状态, Graph 节点将模型输出规范化为结构化 JSON, 再由工具函数转化为 Gmail/Calendar API 请求或本地工作流事件。SQLite 保存工作项、草稿、Proposal、Authorization、Action Event、设置、联系人和记忆; data/ 目录由程序运行时自动创建并被 .gitignore 忽略; Google OAuth 凭据和 API Key 仅在本机配置, 不随仓库分发。

四、关键实现与代码展示

4.1 Agent 核心循环

代码片段 1: Supervisor 节点与下游 Agent 路由

```
def supervisor_agent(state: AssistantState) -> dict[str, Any]:
    """Supervisor Agent: 调用模型理解意图并产生路由计划。"""
    prompt = load_prompt("supervisor_agent")
    result = run_llm_agent(
        agent_name="supervisor_agent",
        state=state,
        system_prompt=prompt,
        agent_input=f"请分析本轮请求并决定需要哪些 Agent。\\n\\n 上下文: \\n{_json_context(state)}",
        tools=create_tools_for_agent("supervisor_agent"),
        max_iterations=1,
    )
    parsed_plan = _parse_supervisor_json(result.content)
```



```

route_agents = _normalize_route_agents(parsed_plan.get("route_agents"))
if not route_agents:
    route_agents = _route_agents_from_message(state, result.content)
# 确认类请求直接交给 Gate/Executor, 避免专业 Agent 再次生成或修改草稿。
if (
    _is_email_send_confirmation(state.get("user_message", ""), state)

```

上述节点将会话状态、用户输入和 Prompt 注入 LLM 调用, 并把模型输出解析为可路由的结构化结果。下游邮件 Agent、日历 Agent、确认节点和执行节点共同组成可审计的任务循环。

4.2 工具定义

代码片段 2: 工具上下文与 Gmail/Calendar 工具函数

```

def set_tool_context(
    *,
    thread_id: str | None = None,
    gmail_client: Any = None,
    calendar_client: Any = None,
    db_session: Any = None,
    user: Any = None,
) -> None:
    """每个请求开始时调用, 注入当前用户上下文 (已创建好的客户端对象)。"""
    global _ctx_thread_id, _ctx_gmail_client, _ctx_calendar_client, _ctx_session, _ctx_user
    _ctx_thread_id = thread_id
    _ctx_gmail_client = gmail_client
    _ctx_calendar_client = calendar_client
    _ctx_session = db_session
    _ctx_user = user

def _get_gmail_client() -> Any | None:

```

工具层通过上下文注入当前用户、会话、Gmail 客户端和 Calendar 客户端, 使 Agent 可以用统一接口调用真实服务, 同时保持工具返回值为 JSON, 便于 Review Gate 和前端展示。

4.3 配置文件

代码片段 3: Google OAuth JSON 与环境变量配置

```

def _try_load_google_oauth_json() -> dict[str, str]:
    """尝试从仓库根目录的 google_oauth.json 读取 OAuth 凭据。

    Google Cloud Console 下载的 OAuth 客户端 JSON 文件格式:
    {"web": {"client_id": "...", "client_secret": "...", "redirect_uris": [...]}}

```

如果文件不存在或格式不对，返回空 `dict`。

```
"""
```

```
json_path = REPO_ROOT / "google_oauth.json"
```

```
if not json_path.exists():
```

```
    return {}
```

- `.env.example` 只保留示例变量，不包含真实 API Key 或 Client Secret。
- `google_oauth.json` 用于本地 OAuth 客户端配置，已被 `.gitignore` 排除。
- Google OAuth 回调地址默认使用 `http://localhost:8000/gmail/auth/callback`，与 README 中的配置说明保持一致。

4.4 AI IDE 使用截图说明

使用场景	说明
需求梳理	使用 AI IDE 读取课程要求，校对 README、.gitignore、项目文档与提交规则。
代码理解	借助 AI IDE 定位 Graph 节点、工具函数、OAuth 配置、SQLite 工作流与前端页面。
实现辅助	围绕多 Agent 编排、工具调用、安全确认和文档整理进行代码阅读与修改。
验证整理	通过命令行检查 Git 状态、项目文档、测试命令和 Word 报告渲染效果。

五、测试与评估

5.1 功能测试

测试项	输入/操作	预期结果	状态
后端单元测试	cd src/backend && python -m pytest	核心服务、Agent 工具和工作流逻辑可通过测试验证	通过
后端静态检查	cd src/backend && python -m ruff check .	代码风格和常见静态问题检查通过	通过
前端生产构建	cd src/frontend && npm run build	Vue 3 前端可完成生产构建	通过
安全确认闭环	邮件/日历写操作生成 Proposal	用户授权前不执行真实外部写操作	通过
凭据保护	检查 Git 跟踪内容	.env、google_oauth.json、data/ 未纳入 Git	通过

5.2 Agent 行为评估

评估重点不是单轮回答是否像聊天，而是 Agent 是否能稳定完成“理解任务 - 补全字段 - 生成 Artifact - Proposal - Authorization - 执行记录”的闭环。项目通过 acceptance_matrix、手工测试脚本和命令行测试覆盖主要路径，并把敏感写操作统一放到授权之后。

功能覆盖与评估结果



评估口径：以 README 与 acceptance_matrix 中的手工验收项、关键路径演示和安全确认逻辑为主。

图 4 主要能力覆盖情况：基于功能验收与手工演示结果整理

5.3 Benchmark 与 Demo

- Benchmark 口径：选取邮件草稿、日历创建、字段缺失、安全确认、上下文记忆等任务作为固定用例。
- Demo 建议：录制本地启动、Google OAuth 授权、创建邮件草稿、确认发送、创建日历事件的完整流程。
- 异常兜底：如演示当天外部 API 不稳定，可使用本地截图、录屏和已保存的手工测试对话作为备份材料。

六、系统升级与扩展

6.1 可扩展架构

当前架构已将前端交互、后端路由、Agent 编排、工具函数和 SQLite 工作流事件分层。扩展 Google Drive、Notion、Slack 或企业内部 OA 时，可新增 OAuth/Token 配置、工具函数和对应 Agent/Prompt，再接入 Supervisor 路由与 Proposal + Authorization 闭环。

6.2 下一阶段计划

阶段	目标	说明
短期	补充自动化测试	为工具函数、确认门和字段校验增加单元测试与集成测试。
中期	完善评估体系	建立固定 Benchmark、失败样例库和 Agent 行为评分表。
中期	增强可观测性	记录关键节点耗时、模型输入输出摘要、工具调用结果和错误类型。
长期	多用户与部署	加入多用户隔离、云端部署、Docker 化和权限管理。

6.3 AI 能力演进路径

- 从单 Agent 执行扩展为多 Agent 协作，进一步区分规划、执行、审查和记忆角色。
- 从静态 Prompt 扩展为可评估、可版本化的 Prompt 与工具策略。
- 从手工验收扩展为自动 Benchmark 与回归评估，降低模型升级带来的行为漂移风险。

七、课程总结

7.1 个人收获

通过本项目，我把大模型从“回答问题的接口”理解为“参与软件工作流的组件”。Agentic AI 的价值不只在生成文本，更在于把模型能力放到真实工具、状态管理、确认机制和工程约束之中。

7.2 工程思维转变

项目开发过程中，最重要的变化是从“让模型一次性做对”转向“让系统在不确定中安全前进”。因此我在设计中加入字段追问、草稿态、确认门、.gitignore 凭据保护和文档化配置，尽量把风险控制在系统边界内。

7.3 对课程的建议

- 建议继续保留从需求、架构、代码到评估的完整交付要求，因为它能促使学生把 AI 工具用于真实工程闭环。
- 建议增加优秀项目的 Demo 参考和评分样例，帮助学生更早理解 SDD、Agent 评估和系统架构图的表达方式。
- 建议课堂中安排更多关于 OAuth、安全配置、API Key 管理和开源提交规范的实践讲解。

参考资料

- 课程大作业要求：企业级应用软件设计与开发（CS599）语雀页面。
- 项目仓库 README.md、docs/architecture.md 与 src/project-docs 下的项目文档。
- Google Gmail API 与 Google Calendar API 官方文档。
- LangGraph / LangChain 相关 Agent 编排文档。